HR System — ChromaDB Vector Database Structure

3 Collections · OpenAI text-embedding-3-small · LangChain + ChromaDB ≥ 1.5.5 · Generated 2026-04-11

■ hr_policies collection	■ company_culture collection	■ job_descriptions collection	All collections use cosine similarity (HNSW default)

Table of Contents

#	Section	Description
1	**Collection Overview**	*All 3 collections at a glance*
2	**hr_policies**	*Leave, attendance & payroll rules*
3	**company_culture**	*Handbook, conduct & values*
4	**job_descriptions**	*Role specs for recruitment screening*
5	**Document & Chunk Schema**	*documents / metadatas / ids structure*
6	**Chunking Configuration**	*RecursiveCharacterTextSplitter settings*
7	**Embedding Configuration**	*OpenAI text-embedding-3-small setup*
8	**ChromaDB Client Configuration**	*HTTP vs local mode, ports, paths*
9	**RAG Query Patterns**	*Tools, k values, retriever setup*
10	**Seeding & Initialization**	*Startup flow, idempotency, file routing*
11	**API Integration Points**	*Endpoints that hit ChromaDB*
12	**PostgreSQL ↔ ChromaDB Link**	*HRPolicy.chroma_doc_id bridge*
13	**Error Handling & Fallbacks**	*Graceful degradation when Chroma offline*
14	**Knowledge Status Endpoint**	*GET /api/knowledge/status response*
15	**Dependencies & Versions**	*Required packages*

1. Collection Overview

Collection Name (Python)	ChromaDB Name	Purpose	Chunking Strategy	Embedding Model	Default k	Source Files
hr_policies	`hr_policies`	Leave, attendance, payroll & general HR policies	RecursiveCharacterTextSplitter chunk_size=600, overlap=80	text-embedding-3-small (OpenAI)	k=3	hr_policies.txt, leave_policy.txt, …
company_culture	`company_culture`	Company handbook, values, code of conduct, disciplinary process	RecursiveCharacterTextSplitter chunk_size=600, overlap=80	text-embedding-3-small (OpenAI)	k=3	hr_handbook.txt, culture*.txt
job_descriptions	`job_descriptions`	Job specifications for AI recruitment screening & culture-fit	RecursiveCharacterTextSplitter chunk_size=600, overlap=80	text-embedding-3-small (OpenAI)	k=2	job*.txt, *description*.txt

HNSW & Storage Configuration

Parameter	Value	Notes
Distance Metric	Cosine Similarity	ChromaDB HNSW default — no explicit configuration set in codebase
Vector Dimensions	1536	Dimension of text-embedding-3-small output vectors
Index Type	HNSW	Hierarchical Navigable Small World graph (ChromaDB default)
Persistence	Docker HTTP / Local file	HTTP: chromadb.HttpClient(host, port=8100) | Local: PersistentClient(path='./chroma_db')

2. Collection: hr_policies

PostgreSQL link: `hr_policies`

Stores chunked HR policy documents. Queried by the Leave Agent before approving/rejecting requests and by the Chat Agent when employees ask policy questions.

Document Schema

Field	Type	Description	Notes	Example Value
ids	String	Auto-generated UUID by ChromaDB	Linked to HRPolicy.chroma_doc_id in PostgreSQL	`e3b0c44298fc1c14...`
documents	String (Text chunk)	Up to 600 characters of raw policy text	Preserves structure: headings, bullet points, section numbers	`ANNUAL LEAVE (AL)` ■■■■■■■■■■■■■■■■■■■■ `Entitlement: 14 working days...`
metadatas	Dict	file (String) — source filename	Additional fields can be added at index time via metadata param	`{"file": "leave_policy.txt"}`
embeddings	List[float] len=1536)	OpenAI text-embedding-3-small vector	Stored internally by ChromaDB — not directly accessible in app code	`[0.012, -0.034, …]`

Metadata Fields Detail

Field	Type	Nullable	Description	Usage
file	String	NOT NULL	Source filename. e.g. "leave_policy.txt", "attendance_policy.txt"	Used to identify which policy document a chunk came from

Leave Types Stored in hr_policies Collection

Code	Leave Type	Max Days/Year	Notice Required	Carry-Over	Document Required	Paid
AL	Annual Leave	14	3 working days	Up to 7 days	No	Yes
SL	Sick Leave	7	Same day	Not allowed	Yes, if >2 days	Yes
CL	Casual Leave	7	1 day	Not allowed	No	Yes
ML	Maternity Leave	84	4 weeks	N/A	Yes	Yes (female)
PL	Paternity Leave	3	1 week	N/A	No	Yes (male)
NPL	No-Pay Leave	—	1 week	Not allowed	No	No

Query Tools Using This Collection

Tool / Function	k	Used By	Example Call
`search_hr_policy(query)`	k=3	Chat Agent, Leave Agent	search_hr_policy("how many annual leave days do I get?")
`get_leave_type_policy(code)`	k=3	Leave Agent	get_leave_type_policy("AL") → query: "AL leave policy entitlement rules conditions"

Tool / Function	k	Used By	Example Call
`_check_policy_rag(code, days)`	k=3	Leave API endpoint	_check_policy_rag("SL", 3) → returns policy text or hardcoded fallback

3. Collection: company_culture

PostgreSQL link: `hr_policies (category=CONDUCT/GENERAL)`

Stores company handbook chunks. Queried when employees ask about workplace culture, conduct rules, disciplinary process, resignation policy, or dress code.

Document Schema

Field	Type	Description	Notes	Example Value
ids	String	Auto-generated UUID by ChromaDB	No explicit PostgreSQL link for culture chunks — managed separately	`7f3a1b9c4d2e…`
documents	String (Text chunk)	Up to 600 characters of handbook text	Covers values, conduct, dress code, digital policy, disciplinary steps	`DISCIPLINARY PROCESS Level 1 — Verbal Warning Issued for: minor policy violations…`
metadatas	Dict	file (String) — source filename	Routing: filenames containing 'handbook', 'culture', or 'conduct' go here	`{"file": "hr_handbook.txt"}`
embeddings	List[float] (len=1536)	OpenAI text-embedding-3-small vector	Stored internally by ChromaDB	`[−0.021, 0.098, …]`

Topics Indexed in company_culture

Topic	Content Summary	Notes
Company Values	Integrity, Innovation, Teamwork, Excellence, Respect	Core values stated in hr_handbook.txt
Code of Conduct	Professionalism, respect, confidentiality, conflict of interest	General conduct rules for all employees
Dress Code	Business casual Mon-Thu, casual Fri, formal for client visits	Uniform / appearance guidelines
Digital Conduct	Email, internet, social media, data security policies	IT and digital usage guidelines
Disciplinary Process	Level 1: Verbal Warning → Level 2: Written Warning → Level 3: Final Warning → Level 4: Termination	Progressive discipline framework
Probation Policy	6-month probation for all new hires; review at end of period	New employee onboarding rules
Resignation & Notice	1 month notice required; 2 weeks for less than 1 year service	Offboarding requirements
Health & Safety	Emergency procedures, fire drills, incident reporting	Workplace safety obligations
Recruitment Process	8-step process: Job posting → Applications → Screening → AI Interview → Shortlist → HR Interview → Offer → Onboarding	Hiring pipeline overview
Performance Management	Monthly check-ins, quarterly reviews, annual appraisal	Review cycle and KPI framework

Query Tools Using This Collection

Tool / Function	k	Used By	Example Query
`search_company_culture(query)`	k=3	Chat Agent	"What is the dress code?" / "What happens in a disciplinary process?"
`search_hr_policy(query)`	k=3	Chat Agent (fallback)	General policy search may also surface culture chunks

4. Collection: job_descriptions

PostgreSQL link: `job_postings (ai_interview_questions / culture_keywords)`

Stores job specification chunks used by the Recruitment Agent to match candidate resumes, conduct AI interviews, and compute culture-fit scores.

Document Schema

Field	Type	Description	Notes	Example Value
ids	String	Auto-generated UUID by ChromaDB	May link to JobPosting.id in PostgreSQL via metadata field	`9a2f6d1c8b4e…`
documents	String (Text chunk)	Up to 600 characters of job spec text	Contains role title, responsibilities, requirements, culture keywords	`SOFTWARE ENGINEER Requirements: 3+ years Python, REST API design, FastAPI or Django…`
metadatas	Dict	file (String) — source filename	Routing: filenames containing 'job' or 'description' go here	`{"file": "job_software_engineer.txt"}`
embeddings	List[float] (len=1536)	OpenAI text-embedding-3-small vector	Used for resume-to-JD cosine similarity scoring	`[0.043, −0.017, …]`

Recruitment AI Scoring Using This Collection

Score Field	Range	How ChromaDB Is Used	Stored In
ai_resume_score	0–100	Cosine similarity between resume embedding and JD chunks	job_applications.ai_resume_score
ai_interview_score	0–100	Average score of AI interview Q&A; answers vs JD requirements	job_applications.ai_interview_score
ai_culture_fit	0–100	Cosine similarity using culture_keywords from JobPosting	job_applications.ai_culture_fit
ai_overall_score	0–100	Weighted average: resume (40%) + interview (40%) + culture (20%)	job_applications.ai_overall_score

Query Tools Using This Collection

Tool / Function	k	Used By	Example Query
`search_job_description(query)`	k=2	Recruitment Agent	"Software Engineer Python FastAPI" / role title or skill keywords

5 & 6. Document / Chunk Schema and Chunking Configuration

ChromaDB Document Structure (All Collections)

ChromaDB Field	Python Type	Required?	Description	App Usage
ids	List[str]	REQUIRED	One UUID string per chunk. Auto-generated by ChromaDB or provided by LangChain.	Linked to HRPolicy.chroma_doc_id in PostgreSQL (hr_policies only)
documents	List[str]	REQUIRED	The raw text content of each chunk (≤ 600 chars). LangChain stores page_content here.	Used as the text passed to the embedding model and returned to the LLM
metadatas	List[Dict]	OPTIONAL	One dict per chunk. Always contains {'file': }. Extra fields can be added.	Returned alongside documents — LLM uses source metadata in responses
embeddings	List[List[float]]	AUTO	1536-dimensional float vector from text-embedding-3-small. Stored by ChromaDB.	Never directly accessed in app code — ChromaDB handles query-time comparison

Chunking Configuration (RecursiveCharacterTextSplitter)

Parameter	Value	Unit	Description	Rationale
chunk_size	`600`	Characters	Maximum characters per chunk. Applies to ALL three collections.	Balances context richness vs retrieval precision
chunk_overlap	`80`	Characters	Overlap between adjacent chunks to preserve sentence context across boundaries.	~13% overlap ratio — standard for policy documents
Splitter type	`Recursive`	Strategy	Tries \n\n → \n → space → char. Preserves paragraphs where possible.	Better than fixed-size for structured policy text
File loaders	`TextLoader / PyPDFLoader`	Per file type	TextLoader for .txt (UTF-8). PyPDFLoader for .pdf files.	Both return List[Document] with page_content and metadata
Add method	`store.add_documents(chunks)`	LangChain API	LangChain Chroma wrapper embeds and inserts all chunks in one call.	Triggers OpenAI API call per chunk for embedding generation

7. Embedding Configuration

Parameter	Value	Source	Notes
Model	`text-embedding-3-small`	OpenAI API	Used for ALL three collections. Consistent embedding space across queries.
Vector dimensions	`1536`	Fixed	Output dimension of text-embedding-3-small.
API Key	`OPENAI_API_KEY env var`	Environment	Set via .env file or Docker compose environment block.

Parameter	Value	Source	Notes
LangChain class	`OpenAIEmbeddings(model='text-embedding-3-small')`	langchain-openai	Wraps OpenAI API. Passed as embedding_function to Chroma().
Query embedding	`Same model`	At query time	Queries are embedded with the same model before cosine similarity search.

8. ChromaDB Client Configuration

Config Key	Env Variable / Class	Default	Description	Mode
CHROMA_HOST	`CHROMA_HOST env`	`localhost`	Hostname for Docker HTTP mode. Empty string = local mode.	HTTP mode only
CHROMA_PORT	`CHROMA_PORT env`	`8100`	TCP port for the ChromaDB HTTP server in Docker.	HTTP mode only
CHROMA_PERSIST_PATH	`CHROMA_PERSIST_DIR env`	`./chroma_db`	File system path for persistent local storage.	Local mode only
Client class (HTTP)	`chromadb.HttpClient`	—	Used when deployed with Docker Compose. Connects to a standalone Chroma container.	Production
Client class (local)	`chromadb.PersistentClient`	—	Used for local development. Stores data on disk at CHROMA_PERSIST_PATH.	Development

9. RAG Query Patterns

Tool / Function	Collection	k	Return Format / Behaviour	Called By
`search_hr_policy(query)`	`hr_policies`	`3`	Returns top 3 policy chunks as JSON with rank, content, source	Chat Agent, Leave Agent
`search_company_culture(query)`	`company_culture`	`3`	Returns top 3 handbook chunks. Used for conduct / culture questions	Chat Agent
`search_job_description(query)`	`job_descriptions`	`2`	Returns top 2 JD chunks. Used for resume scoring and AI interviews	Recruitment Agent
`get_leave_type_policy(code)`	`hr_policies`	`3`	Builds targeted query: '{code} leave policy entitlement rules conditions'	Leave Agent
`_check_policy_rag(code, days)`	`hr_policies`	`3`	Low-level function with hardcoded fallback if ChromaDB unavailable	Leave API /leave/apply

10. Seeding & Initialization

Step / Parameter	Value / Code	Notes
Entry points	app/main.py → seed_policies() hr_agent_system/main.py → seed_all_policies()	Both called at application startup (FastAPI lifespan)
Idempotency check	`collection.count() > 5 (or > 10)`	Skips seeding if collection already has enough chunks — prevents duplicate inserts on restart
File discovery	`glob('*.txt') + glob('*.pdf')`	Scans: backend/app/hr_agent_system/rag/sample_policies/

Step / Parameter	Value / Code	Notes
Routing logic	filename contains 'handbook'/'culture'/'conduct' → company_culture filename contains 'job'/'description' → job_descriptions everything else → hr_policies	Simple string-match classifier — no ML routing
Currently seeded	leave_policy.txt (~5-6 chunks) → hr_policies hr_handbook.txt (~7-8 chunks) → company_culture	~12-14 total chunks at fresh startup
Metadata added	`doc.metadata["file"] = filename`	Applied to every chunk before add_documents() call

11. API Integration Points

API Endpoint / Agent	Collection Used	ChromaDB Tool Called	Purpose
`POST /api/sessions/{id}/message`	`hr_policies`	`search_hr_policy()`	Employee chat — answers policy questions via RAG + LLM
`POST /api/quick`	`company_culture`	`search_company_handbook()`	Quick one-off chat question without creating a session
`GET /api/knowledge/status`	ALL	`collection.count()`	Returns chunk counts per collection — health-check endpoint
`POST /api/leave/apply`	`hr_policies`	`_check_policy_rag(code, days)`	Checks leave policy before Leave Agent makes approval decision
`Recruitment Agent (internal)`	`job_descriptions`	`search_job_description()`	Used during AI screening stage of job application pipeline

12. PostgreSQL ↔ ChromaDB Link

PostgreSQL Column	Type	Description	Purpose
HRPolicy.chroma_doc_id	String(200), UNIQUE, nullable	Stores the ChromaDB UUID for the first chunk of this policy document	Allows PostgreSQL → ChromaDB lookup to check if a document is already indexed
HRPolicy.is_indexed	Boolean, default=False	Set to True once all chunks have been added to ChromaDB	Used by seeder idempotency check and admin dashboard
HRPolicy.indexed_at	Text (ISO datetime), nullable	Timestamp of when the document was indexed into ChromaDB	Audit trail for policy indexing operations
HRPolicy.chunk_count	Integer, nullable	Number of chunks created from this policy document	Stored for reporting; helps estimate retrieval coverage

13. Error Handling & Fallbacks

Scenario	Handling Mechanism	Fallback Behaviour
Leave policy fallback	_check_policy_rag() wraps ChromaDB call in try/except	If ChromaDB is unavailable, returns hardcoded rules: AL=14 days, SL=7 days, CL=7 days, ML=84 days, PL=3 days
Chat RAG fallback	LangChain retriever exception caught by Chat Agent	Agent responds from LLM training knowledge only, notes policy data may be unavailable
Seeding fallback	seed_policies() wrapped in try/except at startup	Application starts normally even if ChromaDB is not reachable — policies seeded lazily on first use
Collection missing	get_or_create_collection() used everywhere	Collections are created automatically on first access — no manual setup required

14 & 15. Knowledge Status Endpoint & Dependencies

Endpoint	Status	Response Body	Purpose
`GET /api/knowledge/status`	`200 OK`	`{"status":"ok","collections":{"hr_policies":{"loaded":true,"chunks":12},"company_culture":{"loaded":true,"chunks":8},"job_descriptions":{"loaded":true,"chunks":0}}}`	Used by admin dashboard to verify ChromaDB health

Python Package Dependencies

Package	Min Version	Role in ChromaDB Pipeline
`chromadb`	`≥ 1.5.5`	Core vector database client (HTTP and local modes)
`langchain`	`≥ 1.2.10`	Agent framework — tools, chains, runnable interface
`langchain-chroma`	`≥ 1.1.0`	LangChain ↔ ChromaDB integration (Chroma wrapper class)
`langchain-community`	`≥ 0.4.1`	TextLoader, PyPDFLoader, document loaders
`langchain-openai`	`≥ 1.1.11`	OpenAIEmbeddings, ChatOpenAI wrappers
`langchain-text-splitters`	`≥ 1.1.1`	RecursiveCharacterTextSplitter
`openai`	`≥ 2.26.0`	Direct OpenAI API client (embeddings + LLM calls)